Guide Équivalence Algèbre - SQL

Movies (title, releaseYear)
Cinemas (name, city)
Spectators (name, birthYear, city)
IsOn (movie, cinema)
  IsOn[movie] $\subseteq$ Movies[title]
  IsOn[cinema] $\subseteq$ Cinemas[name]

Opinions (spectator, movie, stars)
  domain(stars) $= \{0, 1, 2, 3, 4, 5\}$
  Opinions[spectator] $\subseteq$ Spectators[name]
  Opinions[movie] $\subseteq$ Movies[title]
Frequents (spectator, cinema)
  Frequents[spectator] $\subseteq$ Spectators[name]
  Frequents[cinema] $\subseteq$ Cinemas[name]

	Algèbre	**SQL**
Proj.	$\pi_{\text{movie}}(\text{IsOn})$	`SELECT DISTINCT movie FROM IsOn;`

Proj. + Sel.	$\pi_{\text{movie}}(\sigma_{\text{cinema="Hoyts"}}(\text{IsOn}))$	`SELECT movie`
		`FROM IsOn`
		`WHERE cinema="Hoyts";`

Proj. + Nat. Join	$\pi_{\text{movie}}(\text{IsOn} \bowtie \text{Opinions})$	`SELECT movie` `FROM IsOn JOIN Opinions USING (movie);`

Proj. + Sel. + Nat. Join	$\pi_{\text{movie}}(\sigma_{\text{cinema="Hoyts"}}(\text{IsOn}) \bowtie \text{Opinions})$	`SELECT DISTINCT movie`
		`FROM IsOn JOIN Opinions USING (movie)`
		`WHERE cinema='Hoyts';`

Proj. + Theta Join	$\pi_{\text{title, releaseYear}}(\text{IsOn} \bowtie_{\text{movie = title}} \text{Movies})$	`SELECT DISTINCT title, releaseYear` `FROM IsOn JOIN Movies ON (movie=title);`

Prod. Cart.	$\text{Spectators} \times \text{Cinemas}$	`SELECT * FROM`
		`Spectators CROSS JOIN Cinemas;`
		`------------------------`
		`SELECT * FROM Spectators, Cinemas;`

Renommage Att.	$\text{Cinemas (n,c)} \longleftarrow \pi_{\text{name,city}}(\text{Cinemas})$	`SELECT name AS n, city AS c` `FROM Cinemas;`

Renommage Rel.	$\text{C1} \longleftarrow \text{Cinemas};$
	$\text{C2} \longleftarrow \text{Cinemas};$
	$\pi_{\text{C1.name, C2.name}}(\text{C1} \bowtie_{\text{C1.name} < \text{C2.name} \wedge \text{C1.city=C2.city}} \text{C2})$
		`SELECT C1.name, C2.name`
		`FROM Cinemas C1 JOIN Cinemas C2 ON`
		`(C1.name < C2.name AND`
		`C1.city = C2.city);`

Intersection	$\text{R (cinema)} \longleftarrow \pi_{\text{name}}(\text{Cinemas})$
	/ renommage */*
	$\text{R} \cap \pi_{\text{cinema}}(\text{IsOn})$
		`SELECT name AS cinema FROM Cinemas`
		`INTERSECT -- no DISTINCT`
		`SELECT cinema FROM IsOn;`

Union	$\pi_{\text{city}}(\text{Cinemas}) \cup \pi_{\text{city}}(\text{Spectators})$	`SELECT city FROM Cinemas`
		`UNION -- no DISTINCT`
		`SELECT city FROM Spectators;`

Minus	$\text{R (cinema)} \longleftarrow \pi_{\text{name}}(\text{Cinemas})$
	/ renommage */*
	$\text{R} - \pi_{\text{cinema}}(\text{IsOn})$
		`SELECT name AS cinema FROM Cinemas`
		`MINUS -- EXCEPT in SQLite`
		`-- no DISTINCT`
		`SELECT cinema FROM IsOn;`

Division	$\text{R (cinema)} \longleftarrow \pi_{\text{name}}(\text{Cinemas});$ $\pi_{\text{spectator, cinema}}(\text{Frequents}) \,/\, \pi_{\text{cinema}}(\text{R})$	Pas de division en SQL. On peut le reproduire. *Regarder patrons*.
Aggregations	Pas d'operations d'aggregation. **Attention** : on peut reproduire certaines comme le MAX et le MIN. *Regarder patrons*.	`COUNT(), MIN(), MAX(), AVG(), SUM()`
Partition	Pas de partition. *Regarder patrons*.	`GROUP BY`

Lang.	**Patron**
	Couples de spectateurs différents qui fréquentent le MÊME cinéma.
Algèbre	F1 $\longleftarrow$ Frequents ; F2 $\longleftarrow$ Frequents $\pi_{\text{F1.spectator, F2.spectator}}(\text{F1} \bowtie_{\text{F1.spectator} \neq \text{F2.spectator} \wedge \text{F1.cinema=F2.cinema}} \text{F2})$ On peut faire moins que (<) pour filtrer les couples symétriques (Ex. <Marie, Adrian>, <Adrian, Marie>) (Pour Algèbre et SQL)

SQL	`SELECT F1.spectator, F2.spectator`
	`FROM Frequents F1 JOIN Frequents F2 ON`
	`(F1.spectator<F2.spectator AND F1.cinema=F2.cinema);`

	Spectateurs qui fréquentent UN ET UN SEUL cinéma
Algèbre	F1 $\longleftarrow$ Frequents ; F2 $\longleftarrow$ Frequents ; AuMoins2 $\longleftarrow \pi_{\text{F1.spectator}}(\text{F1} \bowtie_{\text{F1.spectator} = \text{F2.spectator} \wedge \text{F1.cinema} \neq \text{F2.cinema}} \text{F2})$ Exactement1 $\longleftarrow \pi_{\text{spectator}}(\text{F1})$ - AuMoins2

SQL	`SELECT spectator`
	`FROM Frequents`
	`GROUP BY spectator`
	`HAVING COUNT (cinema) = 1;`
	
	`SELECT spectator`
	`FROM Frequents`
	`MINUS`
	`SELECT F1.spectator`
	`FROM Frequents F1 JOIN Frequents F2 ON`
	`(F1.spectator = F2.spectator AND F1.cinema<>F2.cinema);`

	Le film LE PLUS ancien
Algèbre	Sans l'opérateur de division : M1 $\longleftarrow$ Movies ; M2 $\longleftarrow$ Movies PlusAncien $\longleftarrow \pi_{\text{title}}(\text{M1})$ - $\pi_{\text{M1.title}}(\text{M1} \bowtie_{\text{M1.releaseYear} > \text{M2.releaseYear}} \text{M2})$ Avec l'opérateur de **division**, on introduit une relation (T1, T2), telle que <x, y> $\in$ PlusAncienOuEgal $\iff$ le film x est sorti avant ou en même temps que y. La solution est alors : PlusAncienOuEgal(T1, T2) $\longleftarrow \pi_{\text{M1.title, M2.title}}(\text{M1} \bowtie_{\text{M1.releaseYear} \leq \text{M2.releaseYear}} \text{M2})$ PlusAncienOuEgal / π_{T2}(PlusAncienOuEgal)

SQL	`SELECT title`
	`FROM Movies`
	`WHERE releaseYear IN (SELECT MIN(releaseYear) FROM Movies);`

	Nom des spectateurs qui fréquentent TOUS les cinémas
Algèbre	R (cinema) $\longleftarrow \pi_{\text{name}}(\text{Cinemas})$; $\pi_{\text{spectator, cinema}}(\text{Frequents}) \ / \ \pi_{\text{cinema}}(\text{R})$

SQL	`SELECT spectator`
	`FROM Frequents`
	`GROUP BY spectator`
	`HAVING COUNT(cinema) IN (SELECT COUNT(name) FROM Cinemas);`
	`----------`
	`WITH NbCinemasParSpec AS (SELECT spectator, COUNT(cinema) AS nbCinemas`
	`FROM Frequents`
	`GROUP BY spectator),`
	`MaxCinemas AS (SELECT COUNT(name) AS nbMaxCinemas`
	`FROM Cinemas)`
	`SELECT spectator`
	`FROM NbCinemasParSpec JOIN MaxCinemas ON (nbCinemas=nbMaxCinemas);`